Experiment_No_1

In [1]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully")
print("OpenCV Version:", cv2.__version__)
```

```
Libraries imported successfully
OpenCV Version: 5.0.0
```

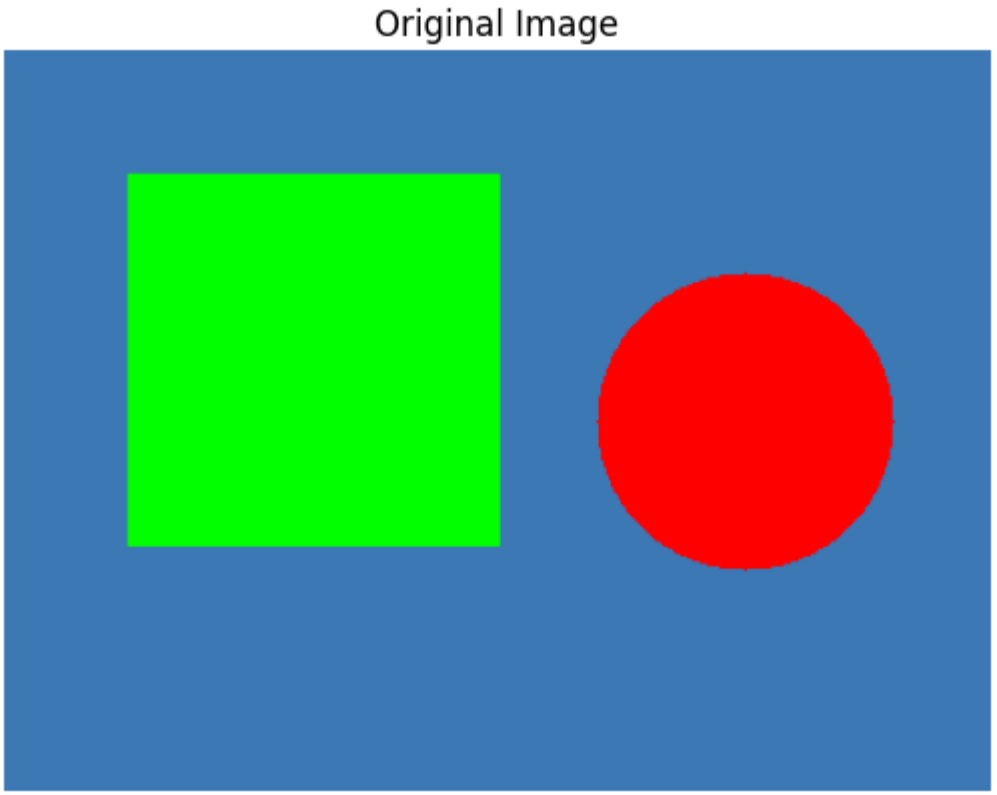
In [2]:

```
# Create a sample color image
image = np.zeros((300, 400, 3), dtype=np.uint8)

image[:] = (180, 120, 60)
cv2.rectangle(image, (50, 50), (200, 200), (0, 255, 0), -1)
cv2.circle(image, (300, 150), 60, (0, 0, 255), -1)

# Convert BGR to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

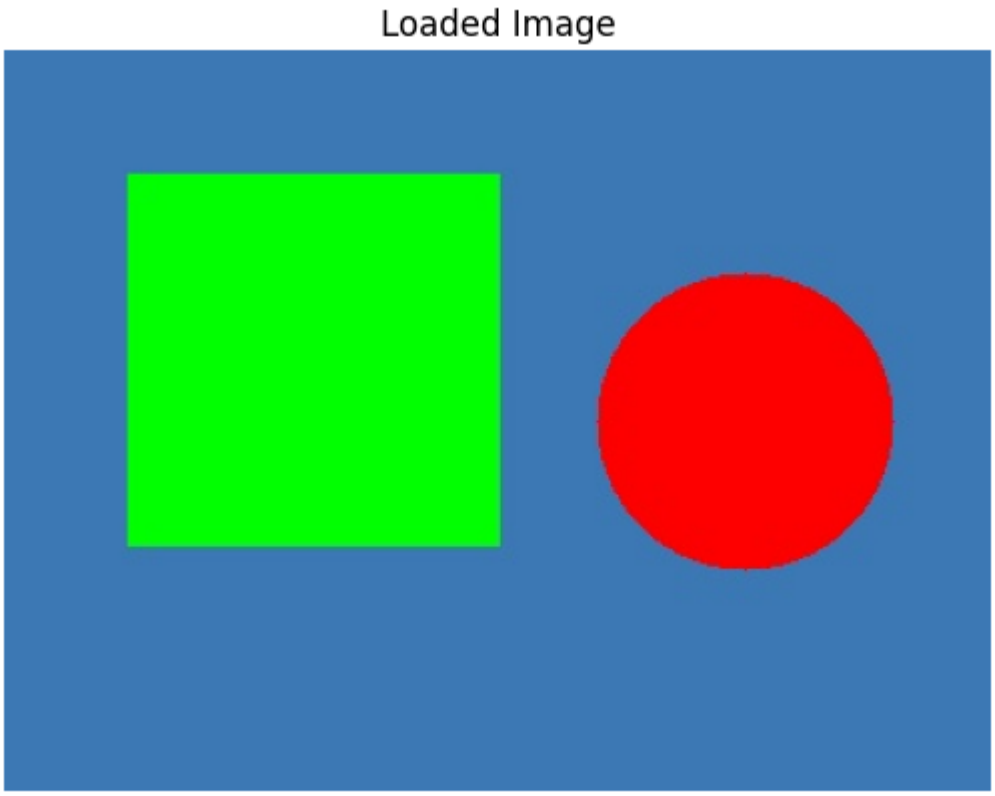


In [3]:

```
# Save image
cv2.imwrite("sample_image.jpg", image)

# Read image
loaded_image = cv2.imread("sample_image.jpg")

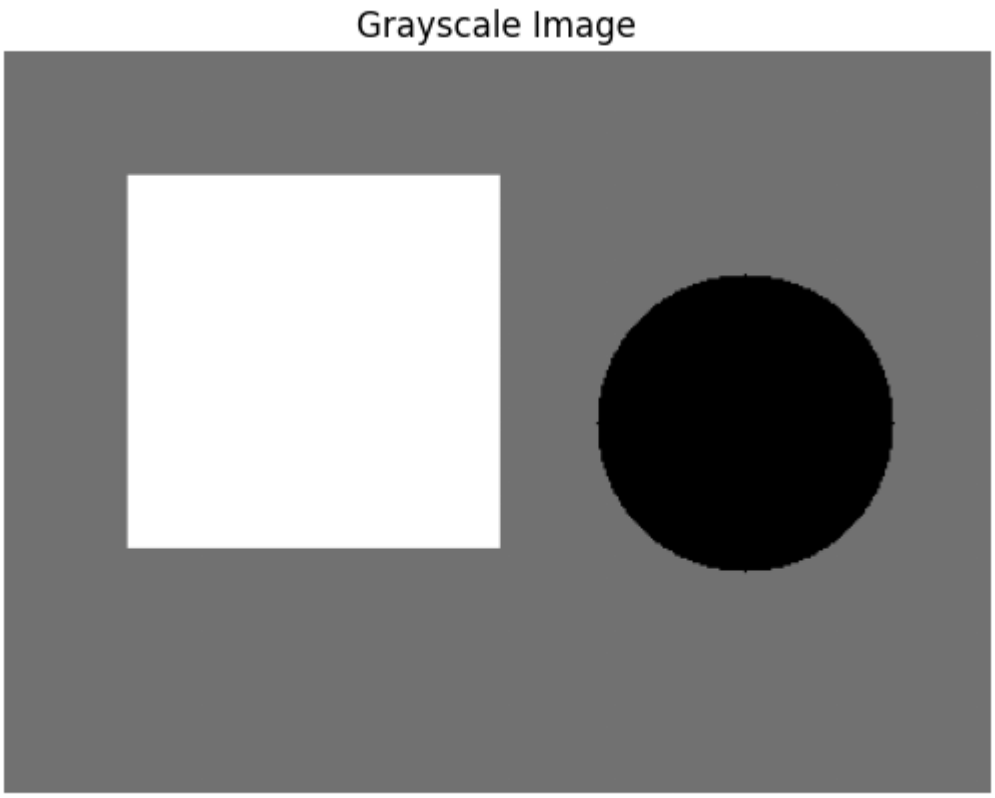
if loaded_image is not None:
    plt.imshow(cv2.cvtColor(loaded_image, cv2.COLOR_BGR2RGB))
    plt.title("Loaded Image")
    plt.axis("off")
    plt.show()
else:
    print("Image not found")
```



In [4]:

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```



In [5]:

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

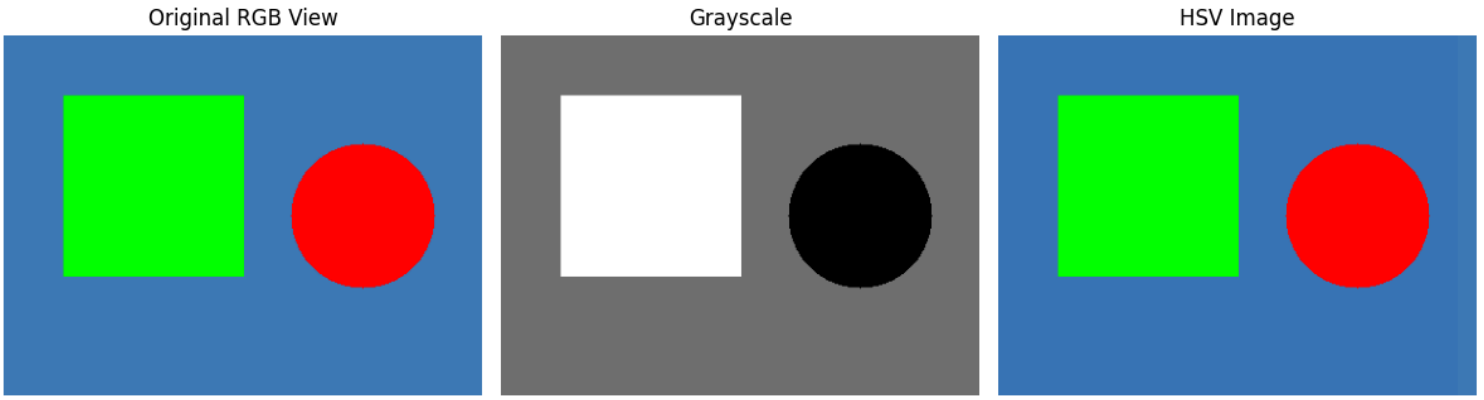
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original RGB View")
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



In [6]:

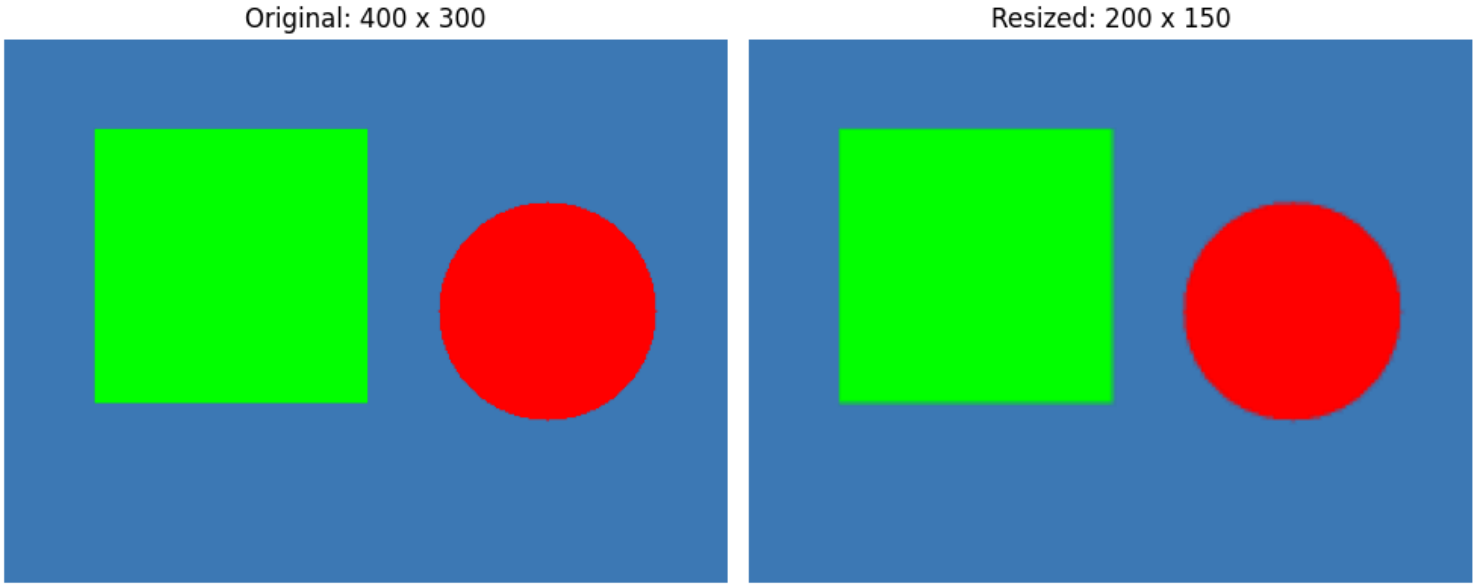
```
resized = cv2.resize(image, (200, 150))

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original: 400 x 300")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized: 200 x 150")
plt.axis("off")

plt.tight_layout()
plt.show()
```



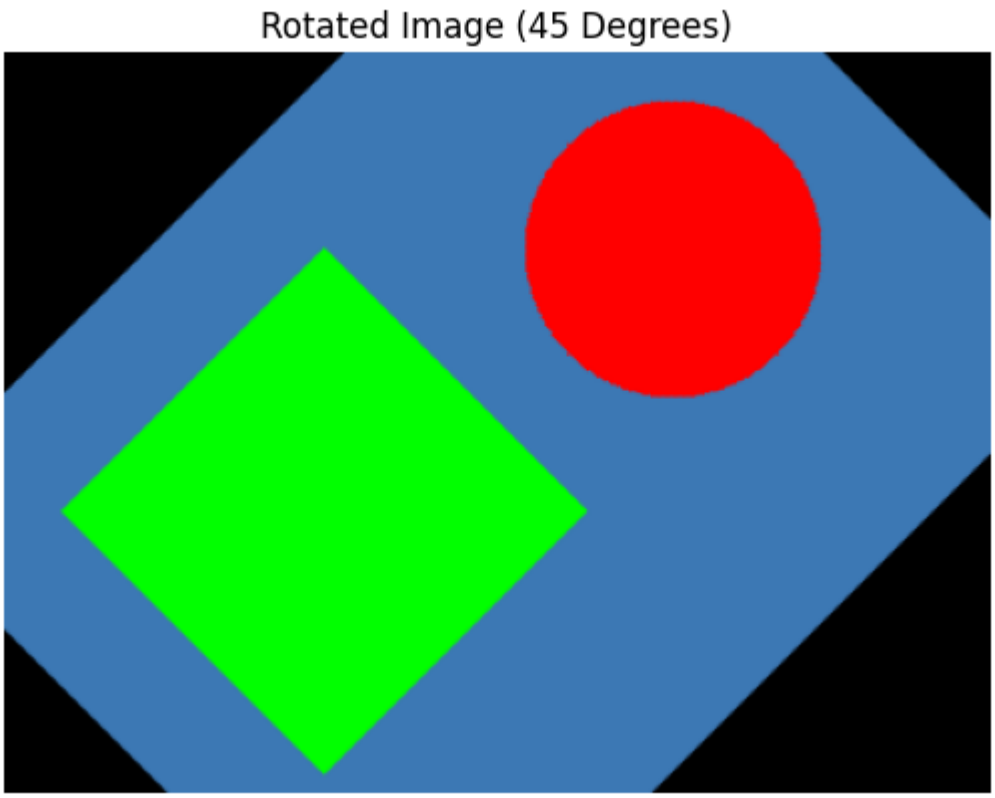
In [7]:

```
height, width = image.shape[:2]

matrix = cv2.getRotationMatrix2D(
    (width // 2, height // 2), 45, 1
)

rotated = cv2.warpAffine(image, matrix, (width, height))
```

```
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image (45 Degrees)")
plt.axis("off")
plt.show()
```



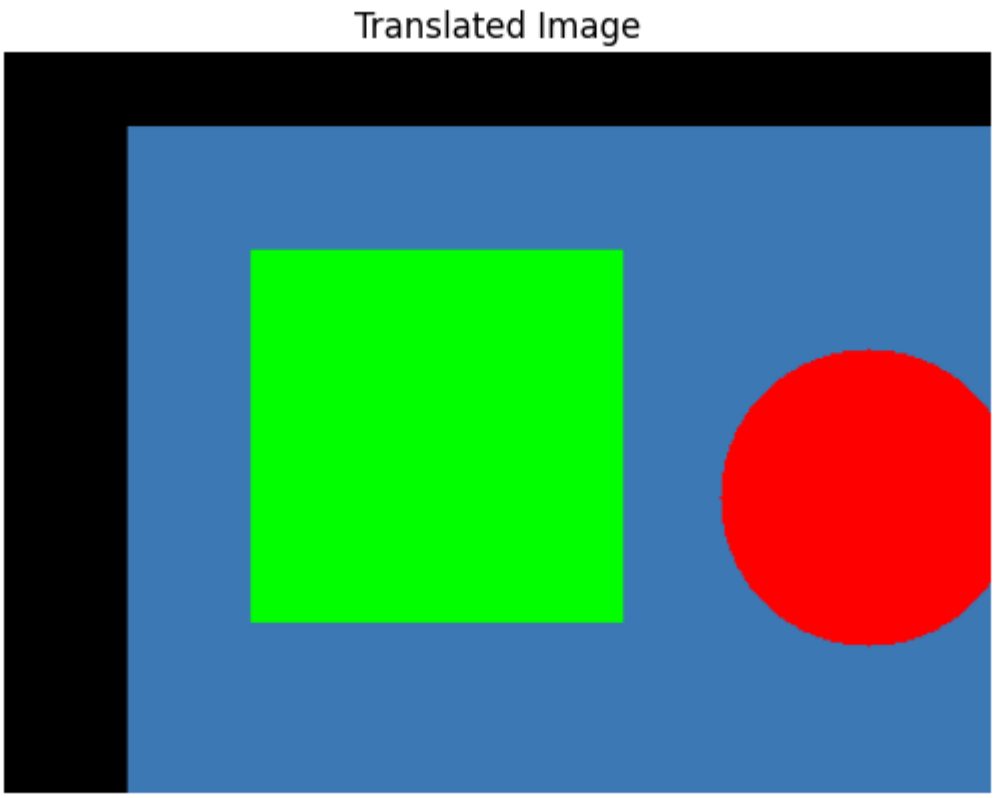
In [8]:

```
height, width = image.shape[:2]

matrix = np.float32([
    [1, 0, 50],
    [0, 1, 30]
])

translated = cv2.warpAffine(image, matrix, (width, height))

plt.imshow(cv2.cvtColor(translated, cv2.COLOR_BGR2RGB))
plt.title("Translated Image")
plt.axis("off")
plt.show()
```



In [9]:

```
# Increase brightness
bright = cv2.convertScaleAbs(image, alpha=1.2, beta=40)

# Increase contrast
contrast = cv2.convertScaleAbs(image, alpha=1.5, beta=0)

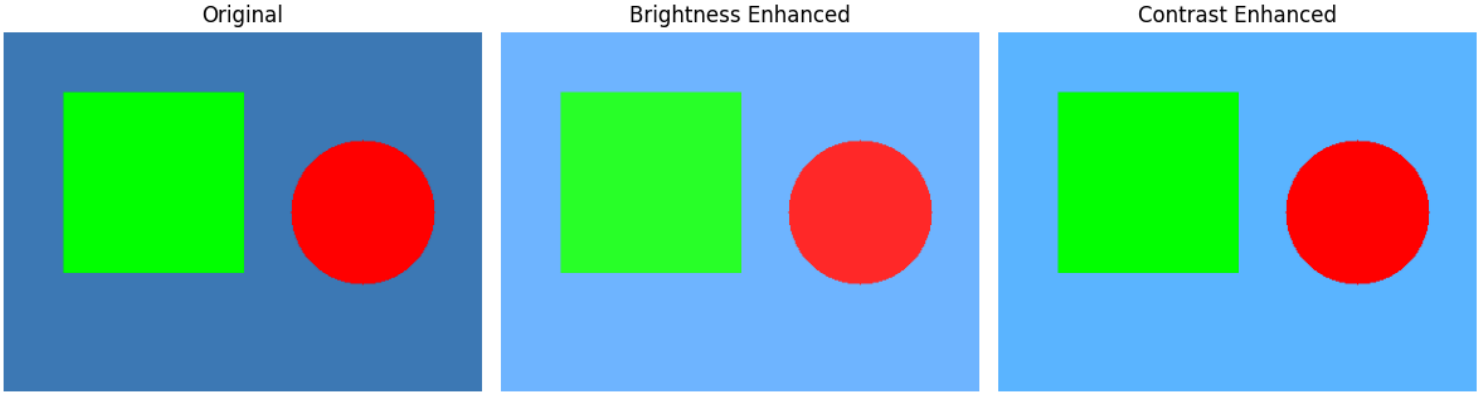
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(bright, cv2.COLOR_BGR2RGB))
plt.title("Brightness Enhanced")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(contrast, cv2.COLOR_BGR2RGB))
plt.title("Contrast Enhanced")
plt.axis("off")

plt.tight_layout()
plt.show()
```



In [10]:

```
cv2.imwrite("grayscale_image.jpg", gray)
cv2.imwrite("resized_image.jpg", resized)
cv2.imwrite("rotated_image.jpg", rotated)
```

```
print("All processed images saved successfully!")
```

```
All processed images saved successfully!
```

In [11]:

```
!pip install opencv-python-headless -q

import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully!")
print("OpenCV Version:", cv2.__version__)
```

```
Libraries imported successfully!
OpenCV Version: 5.0.0
```

In [12]:

```
# Create a sample color image
image = np.zeros((300, 400, 3), dtype=np.uint8)

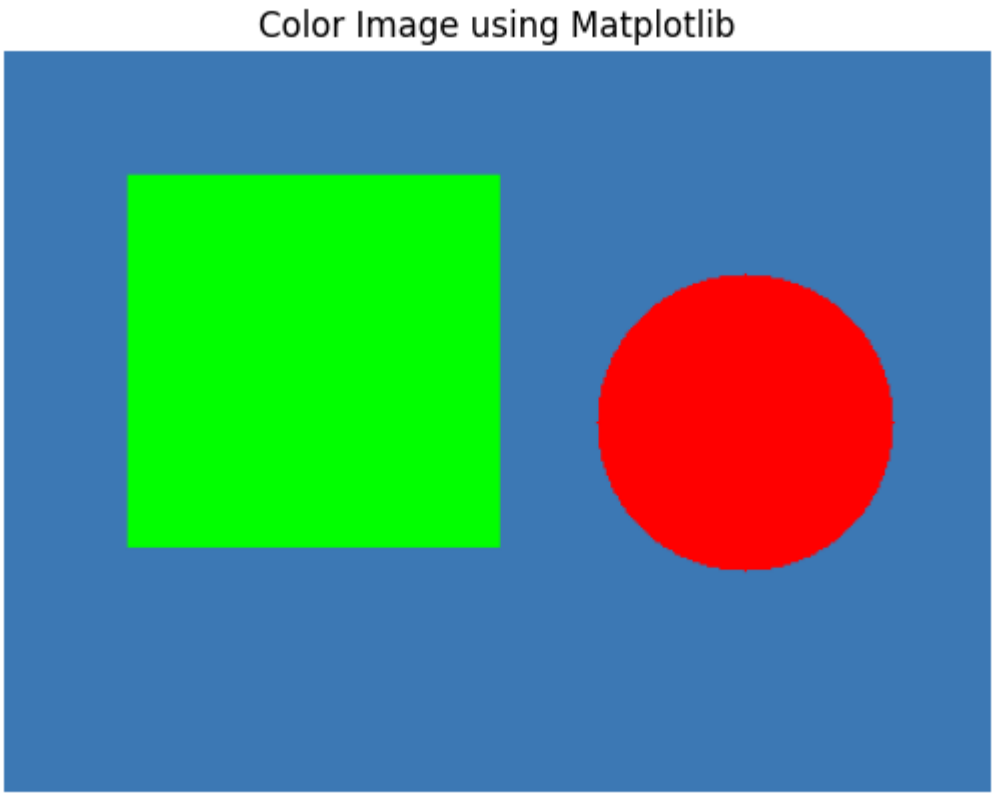
image[:] = (180, 120, 60)
cv2.rectangle(image, (50, 50), (200, 200), (0, 255, 0), -1)
cv2.circle(image, (300, 150), 60, (0, 0, 255), -1)

# Display using OpenCV-compatible conversion
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display using Matplotlib
plt.imshow(image_rgb)
plt.title("Color Image using Matplotlib")
plt.axis("off")
plt.show()

# Save the image
cv2.imwrite("sample_image.png", image)

print("Image created and saved successfully!")
```




```
Image created and saved successfully!
```

In [13]:

```
height, width = image.shape[:2]
channels = image.shape[2]

print("Image Dimensions:", image.shape)
print("Image Width:", width, "pixels")
print("Image Height:", height, "pixels")
print("Resolution:", width, "x", height)
```

```
print("Number of Channels:", channels)
print("Data Type:", image.dtype)
print("Total Pixels:", height * width)
```

```
Image Dimensions: (300, 400, 3)
Image Width: 400 pixels
Image Height: 300 pixels
Resolution: 400 x 300
Number of Channels: 3
Data Type: uint8
Total Pixels: 120000
```

In [14]:

```
# Save in different formats
cv2.imwrite("sample_image.jpg", image)
cv2.imwrite("sample_image.png", image)

# Read both images
jpeg_image = cv2.imread("sample_image.jpg")
png_image = cv2.imread("sample_image.png")

print("JPEG image shape:", jpeg_image.shape)
print("PNG image shape:", png_image.shape)

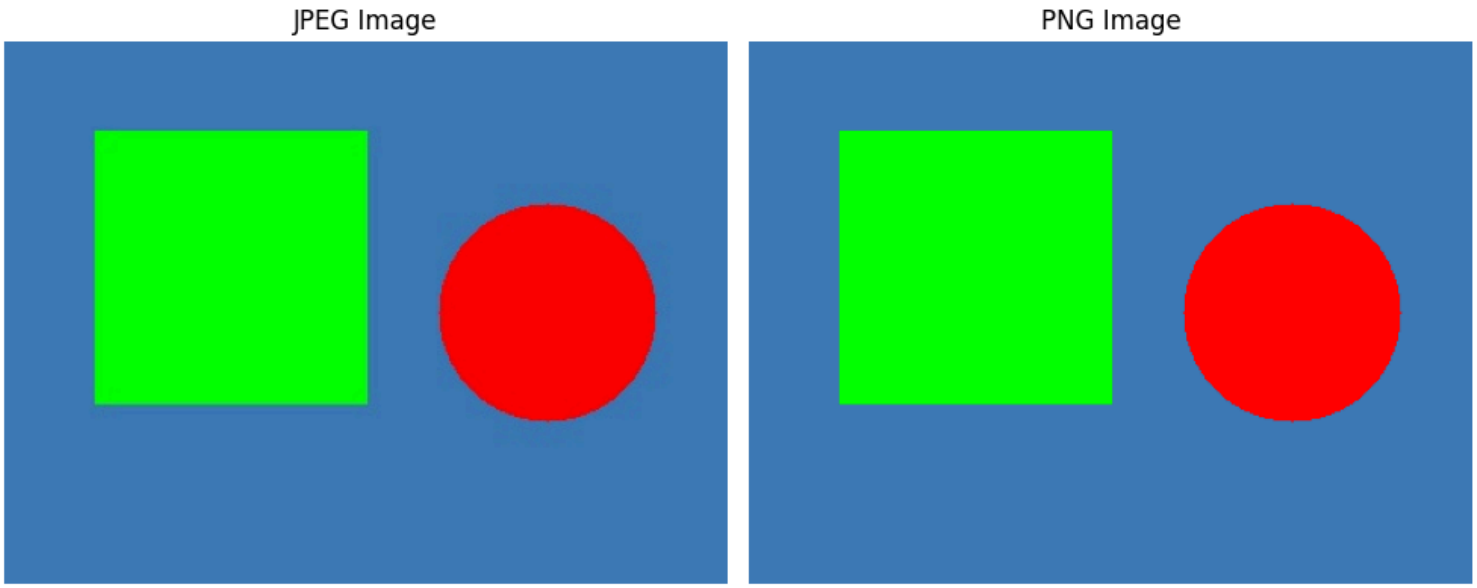
# Display comparison
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(jpeg_image, cv2.COLOR_BGR2RGB))
plt.title("JPEG Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(png_image, cv2.COLOR_BGR2RGB))
plt.title("PNG Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
JPEG image shape: (300, 400, 3)
PNG image shape: (300, 400, 3)
```



In [15]:

```
# Convert to different color spaces
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

# Convert HSV and LAB for correct Matplotlib display
hsv_rgb = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)
lab_rgb = cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)

# Display all color spaces
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original BGR Image")
plt.axis("off")

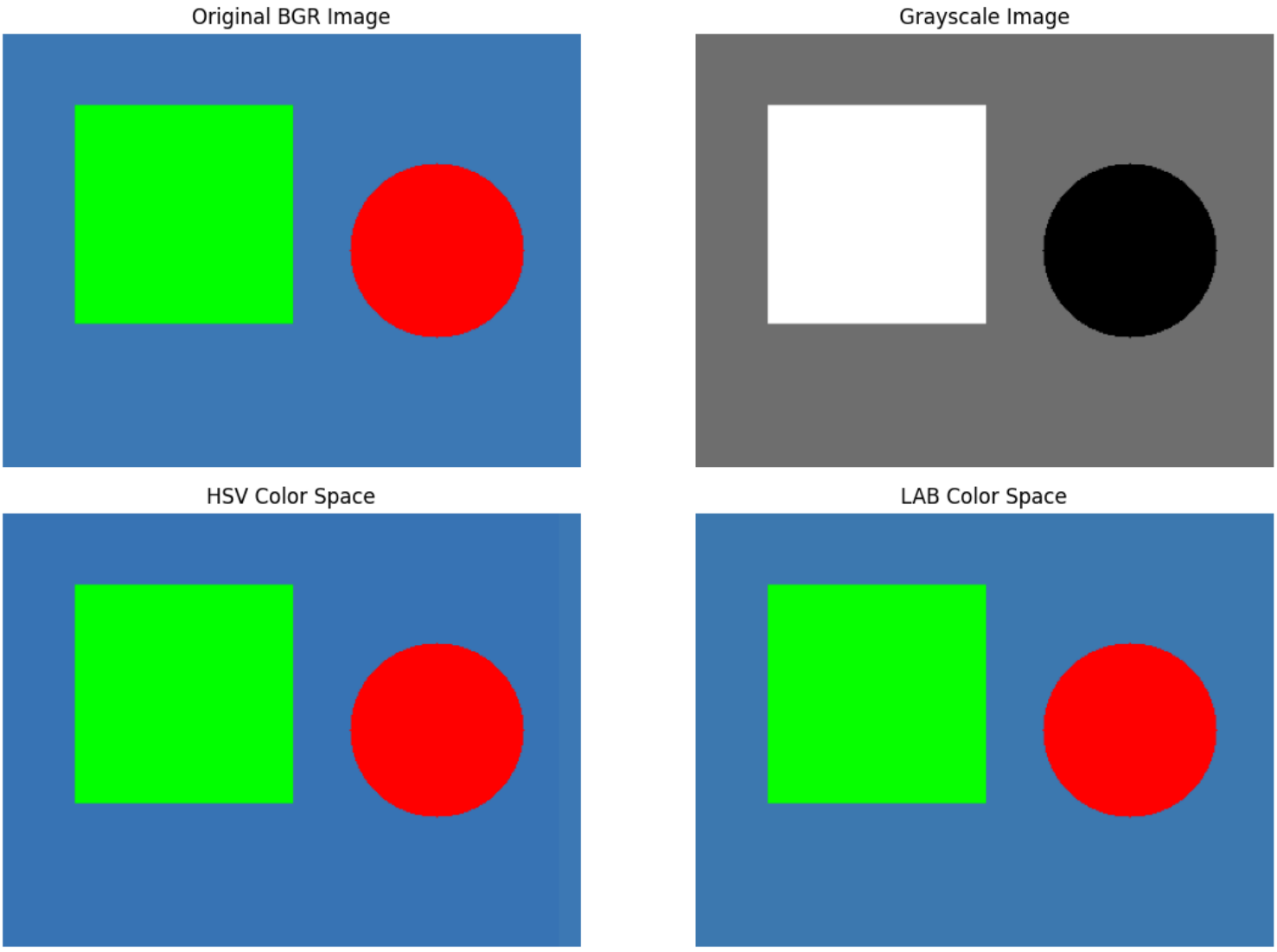
plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(hsv_rgb)
plt.title("HSV Color Space")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(lab_rgb)
plt.title("LAB Color Space")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Color space conversion completed!")
```




```
Color space conversion completed!
```

In [16]:

```
# Resize image
resized = cv2.resize(image, (200, 150))

# Rotate image by 45 degrees
height, width = image.shape[:2]
matrix = cv2.getRotationMatrix2D(
    (width // 2, height // 2), 45, 1
)
rotated = cv2.warpAffine(image, matrix, (width, height))
```

```
# Horizontal flip
horizontal_flip = cv2.flip(image, 1)

# Vertical flip
vertical_flip = cv2.flip(image, 0)

# Display results
plt.figure(figsize=(12, 8))

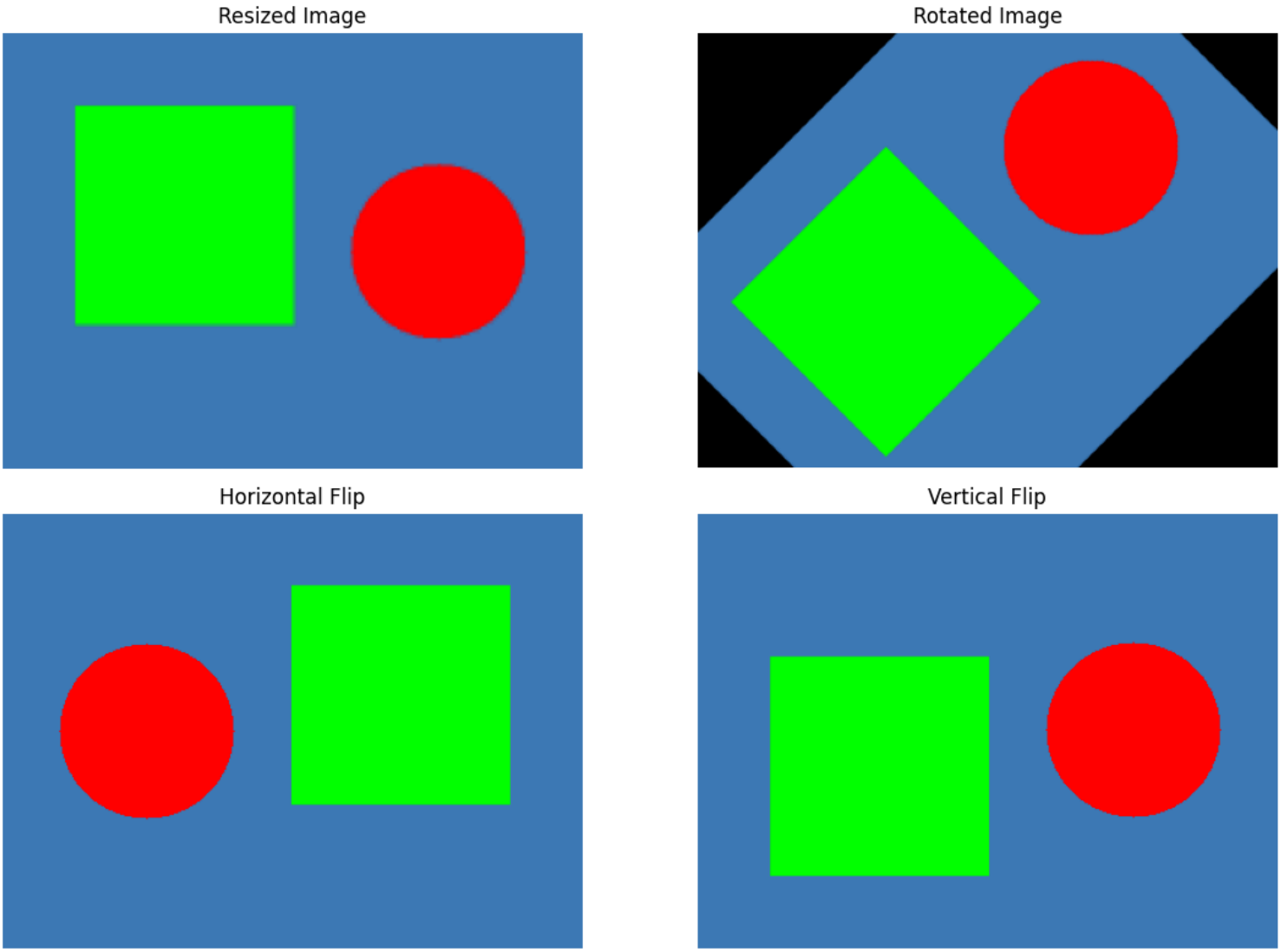
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")

plt.tight_layout()
plt.show()
```



In [17]:

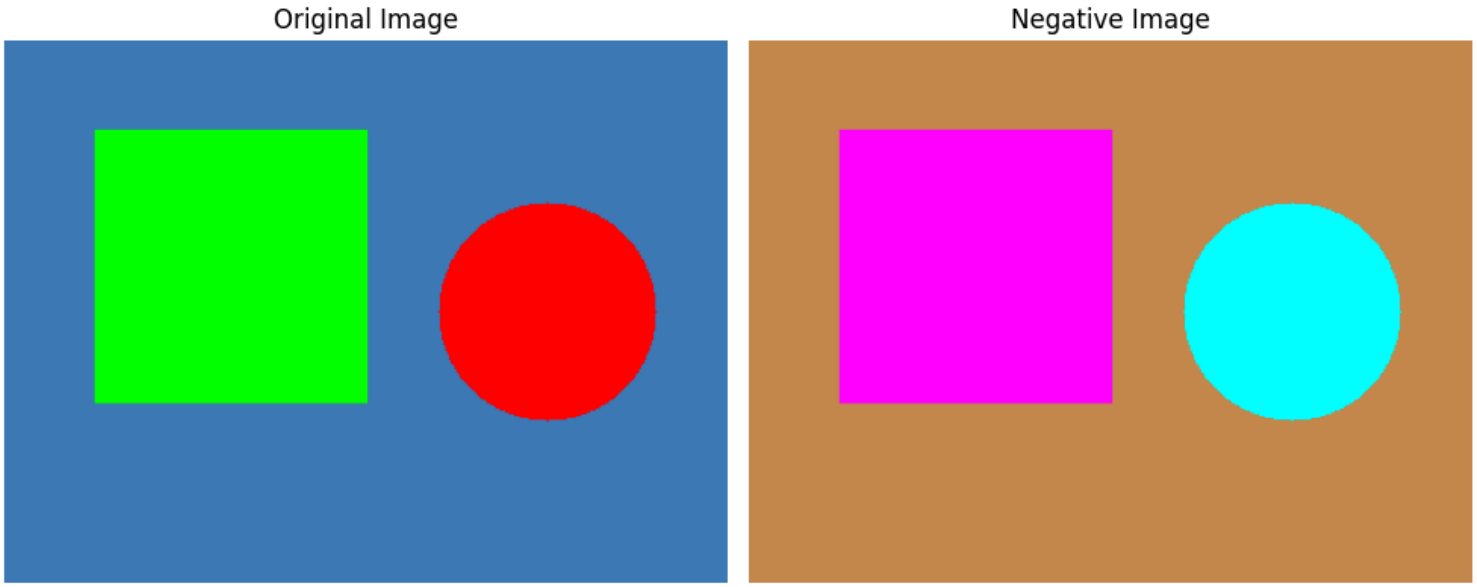
```
negative = 255 - image

plt.figure(figsize=(10, 4))
```

```
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



In [18]:

```
# Crop a region from the image
roi = image[50:200, 50:200]

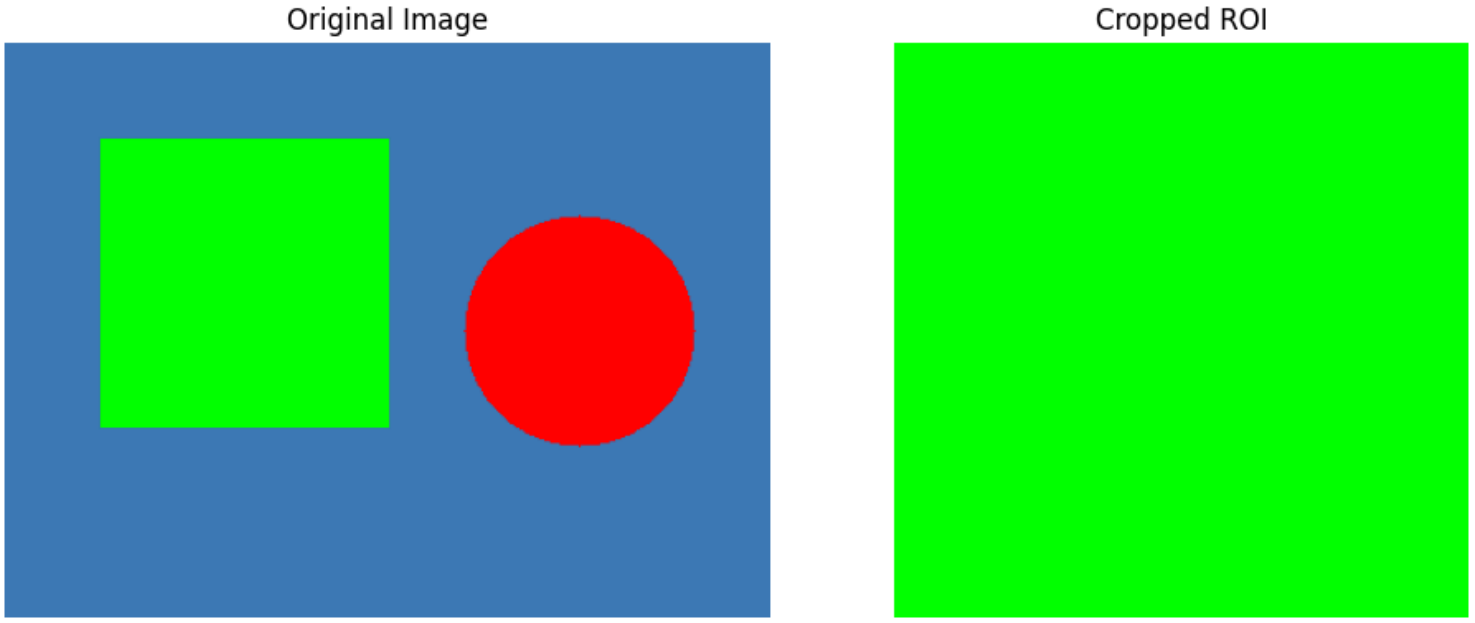
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Cropped ROI")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Original image shape:", image.shape)
print("ROI shape:", roi.shape)
print("ROI pixel count:", roi.shape[0] * roi.shape[1])
```




```
Original image shape: (300, 400, 3)
ROI shape: (150, 150, 3)
ROI pixel count: 22500
```

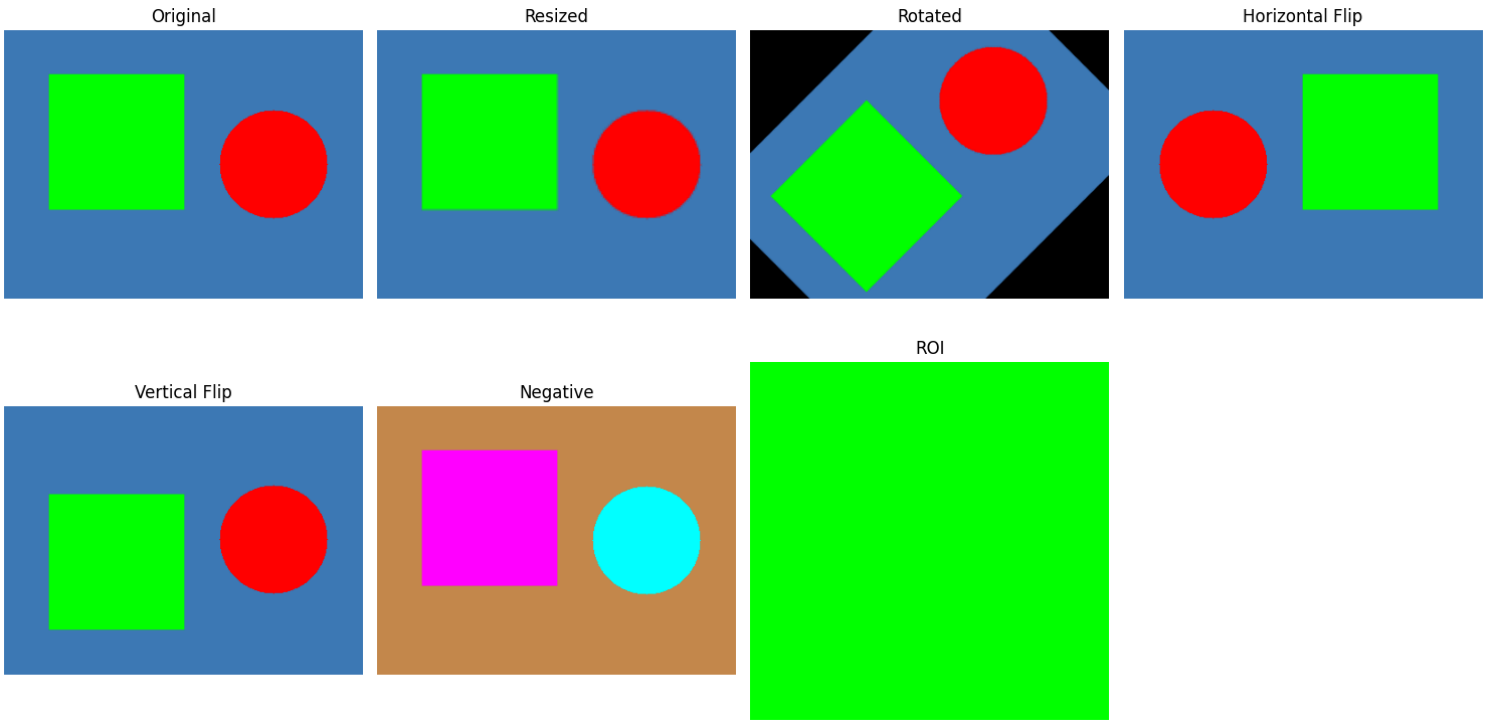
In [19]:

```
plt.figure(figsize=(15, 8))

images = [
    (image, "Original"),
    (resized, "Resized"),
    (rotated, "Rotated"),
    (horizontal_flip, "Horizontal Flip"),
    (vertical_flip, "Vertical Flip"),
    (negative, "Negative"),
    (roi, "ROI")
]

for i, (img, title) in enumerate(images):
    plt.subplot(2, 4, i + 1)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```



In [20]:

```
cv2.imwrite("resized.jpg", resized)
```

```
cv2.imwrite("rotated.jpg", rotated)
cv2.imwrite("horizontal_flip.jpg", horizontal_flip)
cv2.imwrite("vertical_flip.jpg", vertical_flip)
cv2.imwrite("negative.jpg", negative)
cv2.imwrite("roi.jpg", roi)

print("All processed images saved successfully!")
print("Experiment No. 1 completed.")
```

```
All processed images saved successfully!
Experiment No. 1 completed.
```

Part B: Questions and Answers Q1. What is a digital image? Differentiate between grayscale and color images. Answer: A digital image is a two-dimensional array of pixels that represents a visual scene in digital form. Grayscale Image Color Image Contains shades of gray. Contains different colors. Usually has one channel. Usually has three channels. Pixel intensity ranges from 0 to 255 for 8-bit images. Each color channel typically ranges from 0 to 255. Requires less storage. Usually requires more storage. Q2. Explain the difference between RGB, BGR, HSV, and LAB color spaces. Answer:

- RGB: Represents colors using Red, Green, and Blue channels.
- BGR: Uses Blue, Green, and Red channel order. OpenCV commonly uses this order for color images.
- HSV: Represents color using Hue, Saturation, and Value. It is useful for color-based object detection.
- LAB: Contains Lightness (L), green–red (a), and blue–yellow (b) components. It is useful for color analysis and certain enhancement tasks. Q3. What is the purpose of converting an image to grayscale before further processing? Answer: Grayscale conversion reduces a color image to one intensity channel. Its advantages are:
- Reduces computational complexity.
- Requires less memory than a three-channel color image.
- Simplifies edge detection and thresholding.
- Makes certain image processing operations faster. Q4. Explain the concept of image complement (negative image) and mention its practical applications. Answer: An image negative is produced by inverting the pixel intensity values. For an 8-bit grayscale image:

$$s = 255 - r$$

Here, (r) is the original pixel value and (s) is the negative pixel value. For example, black (0) becomes white (255), and white (255) becomes black (0). Applications:

- Enhancing certain details in medical images.
- Examining photographic negatives.
- Highlighting features in some scientific images. Q5. Differentiate between image resizing, cropping, and scaling. Operation Meaning Resizing Changes the image's width and height. Cropping Extracts a selected region and removes the remaining area. Scaling Enlarges or reduces the image relative to its original dimensions. Resizing and scaling are closely related; scaling usually describes applying a size factor, while resizing specifies the desired output dimensions. Q6. What is a Region of Interest (ROI), and why is it important in computer vision? Answer: A Region of Interest (ROI) is a selected portion of an image that contains the object or information we want to analyze. Importance:
- Focuses processing on relevant image regions.
- Reduces unnecessary computation.
- Helps object detection and feature extraction.
- Allows detailed analysis of specific objects. Example: Cropping a vehicle from a traffic image to analyze its shape or number plate. Q7. Why is OpenCV preferred over conventional image processing libraries for computer vision? Answer: OpenCV is widely used because it provides:
- A large collection of image processing and computer vision functions.
- Fast operations implemented in optimized code.
- Support for Python, C++, and other programming languages.
- Tools for image transformations, filtering, object detection, and video processing.
- Integration with NumPy and machine learning workflows.